

PROJECT REPORT

Campus Course & Records Manager (CCRM)

Project Title: Campus Course & Records Manager (CCRM)

Subject: Programming in Java

Technology Used: Java

Project Type: Console-Based Application

Submitted By:

- Name: Prakruti Khosla
- Registration No.: 25BAI10564
- Branch: B.Tech CSE (AI/ML)
- Semester: 3rd

Submitted To:

Faculty Name: Praveen Kumar Tyagi

University: VIT Bhopal University

Academic Year: 2026–27

2.Introduction

The **Campus Course & Records Manager (CCRM)** is a Java-based console application designed to manage academic information in an educational institution.

The system provides functionalities for managing:

- Student records
- Course information
- Student enrollment

- Grades
- Academic records
- Data import and export
- File-based data storage
- Searching and filtering

The project demonstrates important Java programming concepts such as **Object-Oriented Programming, inheritance, abstraction, encapsulation, polymorphism, collections, exception handling, file I/O, Stream API, lambda expressions and design patterns.**

3. Problem Statement

Managing student and course information manually can be time-consuming and error-prone.

Educational institutions need a system that can efficiently:

- Store student information.
- Maintain course details.
- Enroll students into courses.
- Manage grades.
- Search and retrieve records.
- Maintain academic information.
- Import and export records.
- Handle invalid data and errors.

Therefore, the CCRM project provides a centralized **Java-based academic record management system** through a simple command-line interface.

4. Functional Requirements

The functional requirements describe what the system should do.

Student Management

- Add a new student.

- View student details.
- Update student information.
- Delete student records.
- Search for students.
- Maintain student status.

Course Management

- Add courses.
- View available courses.
- Update course information.
- Remove courses.
- Search and filter courses.
- Maintain course capacity and credits.

Enrollment Management

- Enroll students in courses.
- Check enrollment conditions.
- Prevent invalid or duplicate enrollment.
- Maintain enrollment records.
- Track enrolled courses.

Grade Management

- Assign grades to students.
- Update grades.
- Maintain academic records.
- Generate academic information/transcripts.
- Calculate GPA where applicable.

Data Management

- Import data from CSV files.
- Export records to CSV.
- Store application data using files.
- Create backups of important data.

5. Non-Functional Requirements

Performance

- The application should respond quickly to user commands.
- Searching and processing records should be efficient.

Reliability

- The system should prevent invalid operations.
- Errors should be handled properly.

Usability

- The application should provide a simple menu-driven interface.
- Users should receive understandable messages.

Maintainability

- The project should use separate packages for different responsibilities.
- Code should follow object-oriented principles.

Portability

- Since Java runs through the JVM, the application can run on different operating systems with a compatible JDK.

Security

- Input validation should be used to prevent invalid records.
- Application data should be handled carefully.

6. System Architecture

The project follows a **layered/modular architecture**.

Main Components

1. CLI Layer

- Provides interaction between the user and application.
- Accepts commands/input.
- Displays results.

2. Domain Layer

- Represents important entities such as:
 - Student
 - Instructor
 - Course
 - Enrollment
 - Person

3. Service Layer

- Contains the main business logic.
- Handles student management.
- Handles course management.
- Handles enrollment and academic operations.

4. I/O Layer

- Handles file operations.
- Reads and writes CSV files.
- Supports import/export and backup operations.

5. Configuration Layer

- Stores application configuration.
- Provides configurable application settings.

6. Utility Layer

- Contains reusable helper functions such as:
 - Date/time utilities
 - Comparators
 - File utilities

7. Exception Layer

- Contains custom exceptions and error-handling logic.

The repository structure for CCRM-type implementations commonly separates cli, domain, service, io, config, util, and exception packages.

7. Design Diagrams

You need **5 diagrams** according to your teacher's format.

A. Use Case Diagram

Actors:

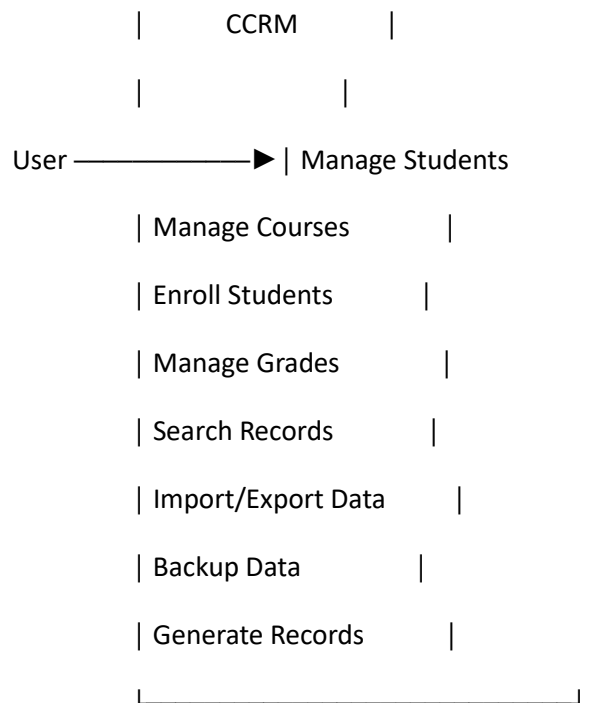
Administrator/User

Main Use Cases:

- Manage Students
- Manage Courses
- Enroll Student
- Manage Grades
- Search Records
- Import Data
- Export Data
- Backup Data
- Generate Academic Records

You can represent it like:





B. Workflow Diagram

START



Launch Application



Display Main Menu



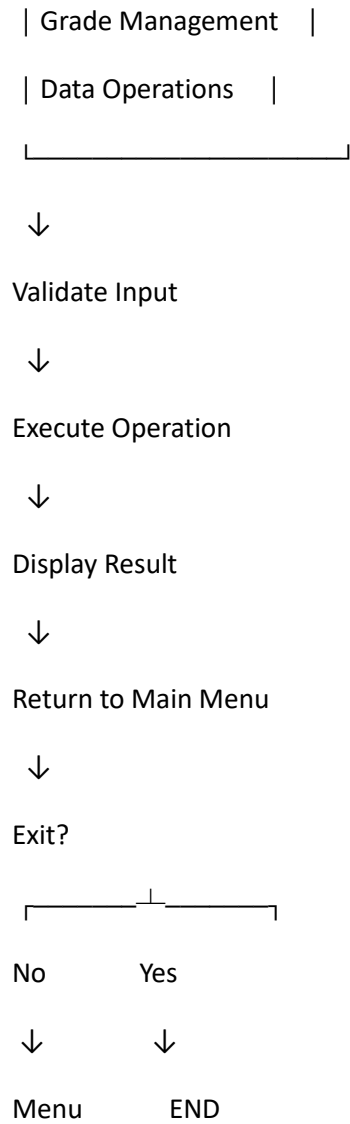
Select Operation



| Student Management |

| Course Management |

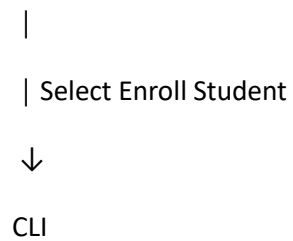
| Enrollment |



C. Sequence Diagram

Example: **Student Enrollment**

User



|

| Send Student & Course Details

↓

Enrollment Service

|

| Validate Student

|

| Validate Course

|

| Check Enrollment Rules

↓

Enrollment

|

| Create Enrollment

↓

Service

|

| Save/Update Record

↓

CLI

|

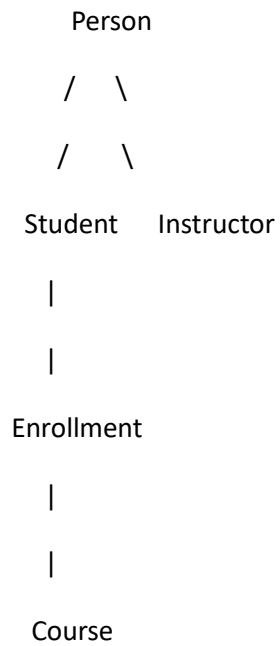
| Display Success Message

↓

User

D. Class/Component Diagram

Main classes/components can be shown as:



CLI

|

↓

Service Layer

|

└─ Student Service

└─ Course Service

└─ Enrollment Service

└─ Transcript Service

|

↓

I/O Layer

|

└─ CSV/File Storage

The CCRM architecture commonly uses inheritance such as Person → Student/Instructor, service classes for business logic, and interfaces such as persistence/search contracts.

E. ER Diagram

Since the project uses file/CSV-based persistence rather than a conventional relational database, you can mention:

ER Diagram is not applicable as the primary storage mechanism is file-based CSV storage.

If your teacher specifically requires an ER diagram, you can still represent the logical relationships:

STUDENT

Student_ID

Name

Email

Status

|

| enrolls

↓

ENROLLMENT

Enrollment_ID

Student_ID

Course_ID

Grade

|

| belongs to



COURSE

Course_ID

Course_Name

Credits

Instructor

Semester

8. Design Decisions & Rationale

Object-Oriented Design

OOP was selected because the project deals with real-world entities such as students, courses and instructors.

Encapsulation

Data members are kept private and accessed using methods.

Reason:

It protects data and controls how information is modified.

Inheritance

A common base class such as Person can be extended by Student and Instructor.

Reason:

It avoids code duplication and represents the relationship between entities.

Abstraction

Abstract classes/interfaces are used to define common behaviour.

Reason:

It hides unnecessary implementation details.

Polymorphism

Services and objects can provide different implementations of common operations.

Reason:

It makes the application flexible and maintainable.

Collections

Collections such as maps/lists are used for managing multiple records.

Stream API

Streams can be used for searching, filtering and processing records efficiently.

NIO.2

Java NIO.2 is used for modern file and directory operations.

Design Patterns

The project demonstrates patterns such as **Singleton** for configuration and **Builder** for object creation in CCRM implementations.

9. Implementation Details

Programming Language

Java

Development Environment

- JDK
- Eclipse/IntelliJ/VS Code
- Command Prompt/Terminal
- GitHub

Main Packages

src/

└─ edu/

└─ ccrm/

└─ cli/

└─ config/

└─ domain/

└─ exception/

└─ io/

└─ service/

└─ util/

Important Java Concepts Used

- Classes and Objects
- Encapsulation
- Inheritance
- Abstraction
- Polymorphism
- Interfaces
- Collections
- Generics
- Lambda Expressions
- Stream API
- Exception Handling
- File I/O
- Date/Time API
- Assertions
- Design Patterns

A comparable CCRM implementation maps these concepts to classes such as Person, Student, service implementations, import/export services and utility classes.

10. Screenshots/results

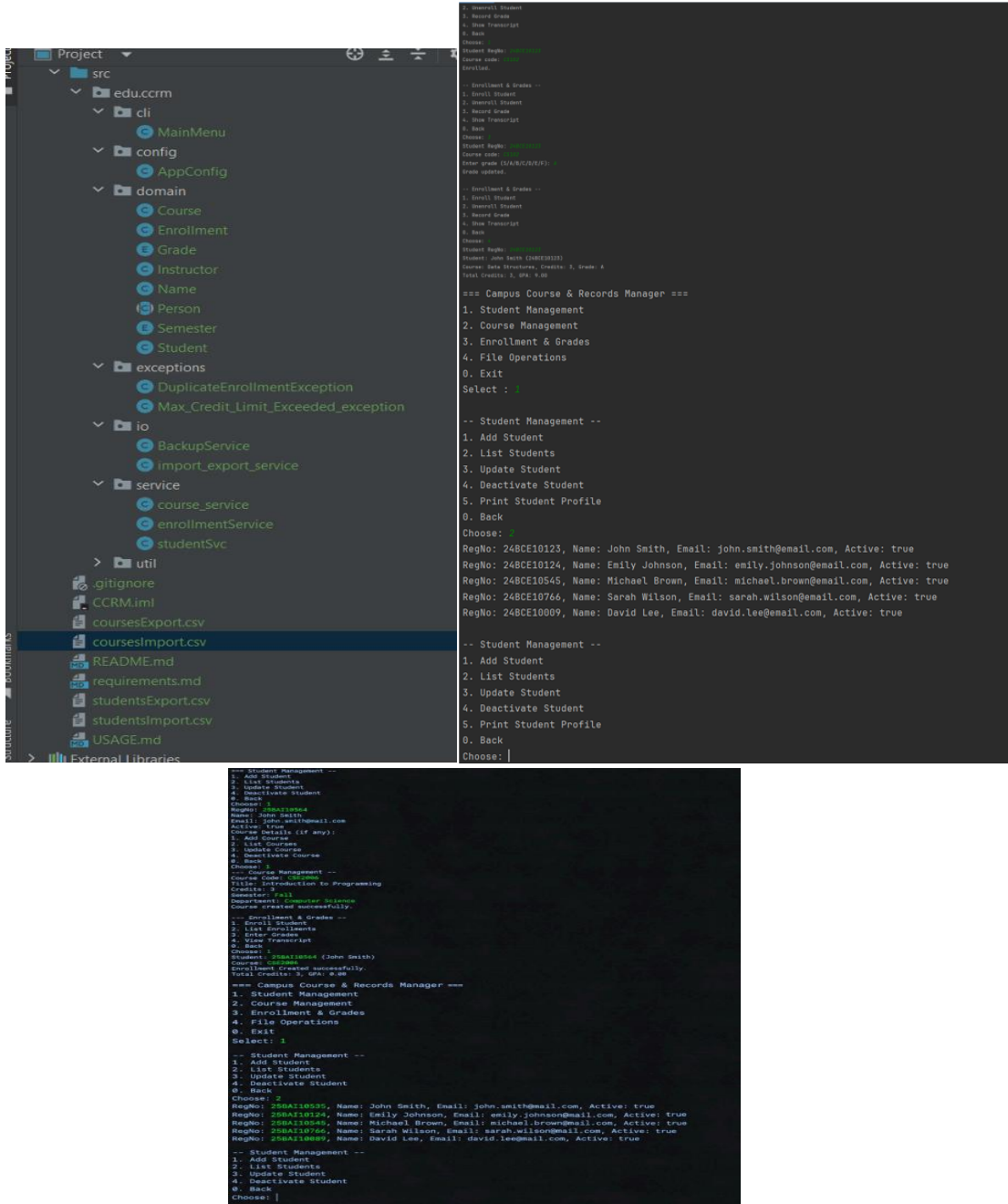
```
C:\Users\adity>java -version
java version "26.0.2.1" 2026-08-18
Java(TM) SE Runtime Environment (build 26.0.2.1+1-7)
Java HotSpot(TM) 64-Bit Server VM (build 26.0.2.1+1-7, mixed mode, sharing)
```

```
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.1.4\lib\idea_rt.jar:45430;-Didea.config.path=C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.1.4\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath ;C:\Users\adity\IdeaProjects\CRM\out\production\CRM edu.core.v1.MainMenu

*** Campus Course & Records Manager ***
1. Student Management
2. Course Management
3. Enrollment & Grades
4. File Operations
0. Exit
Select : 4

-- File Utilities --
1. Import Students
2. Export Students
3. Import Courses
4. Export Courses
5. Backup Data
6. Show Backup Directory Size
0. Back
Choose : 1
Path to CSV: studentsimport.csv
Imported 5 students.

-- File Utilities --
1. Import Students
2. Export Students
3. Import Courses
4. Export Courses
5. Backup Data
6. Show Backup Directory Size
0. Back
```



11. Testing Approach

Testing was performed to verify that each major functionality works correctly.

Functional Testing

Tested:

- Student creation
- Student update
- Student deletion
- Course creation
- Course search
- Student enrollment
- Grade assignment
- CSV import/export
- Backup operation

Input Validation Testing

Checked:

- Empty inputs
- Invalid IDs
- Duplicate records
- Invalid course information
- Invalid enrollment conditions

Exception Testing

The system handles errors using Java exception-handling mechanisms and custom exceptions where required.

Integration Testing

Different modules were tested together, for example:

Student



Enrollment



Course



Grade



Academic Record

Assertions

Java assertions can also be used to verify assumptions and business rules during development. CCRM implementations commonly use assertions for validation and invariants.

12. Challenges Faced

During development, the following challenges can be discussed:

- Designing the project into multiple packages.
- Managing relationships between students, courses and enrollments.
- Maintaining data consistency.
- Handling invalid user input.
- Implementing file-based data storage.
- Reading and writing CSV files.
- Handling exceptions properly.
- Implementing search and filtering operations.
- Understanding and applying OOP concepts.
- Integrating different service modules.
- Debugging errors during execution.

13. Learnings & Key Takeaways

Through this project, we learned:

- Practical implementation of Java OOP.
- How to design a modular Java application.

- How inheritance and abstraction are used in real applications.
- How interfaces improve flexibility.
- How collections store and manage multiple records.
- How Stream API simplifies data processing.
- How Java handles files using NIO.
- How exceptions improve application reliability.
- How CSV files can be used for data persistence.
- How design patterns can improve software structure.
- How to test and debug a Java application.
- How to use GitHub for project management and collaboration.

14. Future Enhancements

The project can be improved by adding:

- **Graphical User Interface (GUI)** using JavaFX or Swing.
- **Web-based interface.**
- **MySQL/PostgreSQL database** instead of CSV files.
- User authentication and role-based access.
- Admin/student separate dashboards.
- Automated email notifications.
- Advanced reports and analytics.
- Attendance management.
- Timetable management.
- Course recommendation system.
- Cloud-based data storage.
- REST API integration.
- Mobile application support.

15. References

You can put:

1. Oracle Java Documentation – Java SE Documentation.
2. Java Platform documentation for OOP, Collections, Streams and NIO.
3. GitHub repository – **Prakrutikhosla-hash**
4. Eclipse IDE Documentation.
5. Java Tutorials and API Documentation.
6. Course material provided for Programming in Java.